

Lec 59 Transformer Architecture: Decoder

Lec 59 Transformer Architecture: Decoder

Lecture	59 / 63 + intro
Duration	35:20
Captions	English (auto-generated)
Words	4,874
Video	if65zXEwIKI
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone, welcome back to the lecture series on fundamentals of generative AI and large language models. In my previous lecture, we have discussed about the transformer architecture mainly the encoder part. In today's session, we will be discussing in detail the decoder part of the transformer architecture. Okay. So the contents of today's session goes like the decoder architecture sub layers inside the decoder encoder decoder connection subword tokenization under that one we have bite pair encoding and word piece tokenization and then training of the transformer. So together with understanding the decoder architecture, we will also touch upon the training and you know the uh tokenizations that are required in the

[00:52] transformer architecture. Okay. So as we all know the transformer is made up of two main components. The encoder that we have already studied in our previous session in um details. So today we will look into the decoder. Once again the decoder is made up of uh six stacked layers of decoder. So decoder one to decoder six. So decoder layer 1 to six. And then each of these sub layer each of these decoder layers have three sub layers. Okay. So in encoder we had two sub layers. In decoder we have three sub layers. What are those sub layers? One is the masked multi head

attention. Encoder decoder attention which is once again a multi head attention and feed forward network.

[01:39] Okay. And this is there for you know all the layers and then uh till the sixth layer. So all the layers have um encoder decoder uh sorry all the layers have these three sub layers. Okay. Now that is the entire decoder here and uh then we will look into the details. So the decoder architecture so we have the multi head masked multi head attention then multi head attention that is coming from encoder decoder attention that is the second uh sub layer and then position wise uh feed forward network.

[02:18] Okay so these are the main three sub layers and then we will see how they are connected or how the information flow happens in through these three uh sub layers. So we have the output embedding going into the decoder. Now this is different from the encoder part because in encoder we have seen the input sequence is going um in the decoder where uh in the encoder whereas in the decoder part the a portion of the output is going into the encoder as an input during the you know uh uh you know inference time. So it is taken the output is taken and put there as the input and then we have output embeddings which are and then the output embeddings and then it is added with the positional encoding and that then the entire final

[03:10] embedding of the output uh token is going into the masked multi head attention. Now we all we have studied in my previous lectures what is this masked multi head attention does. So whenever we are predicting the next word in a sentence, the model should not see the word that is you know there next. So what happens is we mask that portion and the model can see only the previous words. Okay. So that is the case in masked multi head attention.

[03:45] That is what we do. So ensures that each token a target token attends only to the previously generated tokens. Okay. So this is target tokens that means during the training process we have the target you know um uh sentences and then the multi head masked multi head attention masks the portions which are going to come and the model have only access to the previously generated tokens. Okay. And then we have um from here it goes to the add and normalization like the encoder only. And then there is a residual connection same way and residual connection goes between the you know it wraps around from here to the add and nom layer and then from there we get the add and nom layer.

[04:41] It combines the original input with the sub layers output and normalizes it to improve the training stability. So we have seen in our previous lecture also that when the input goes through a residual connection and the residual connection adds the input to the sub layers output and then the entire thing goes through normalization process and this is done to make the training stability better from add and norm then it goes to multi head attention.

[05:17] which is basically an encoder decoder attention between the encoder and the decoder the attention mechanism. It is also a multi head attention and it enables the decoder to attend to the encoder's output because the encoder's output is coming to the decoder. The decoder needs to understand what is there in the encoder's output because the encoder's output will give the contextual understanding of the input words. in the input sequence. So this multi head attention or encoder decoder based attention that enables the decoder to attend to the encoder's output. So which is very important. Okay. And from there we have another add on layer adduh add addition with the residual connection and the normalization after the residual

[06:10] connection and then it goes to position wise position wise feed forward network. Okay. And what does position wise feed forward network does? It independently transforms each token representation to learn richer and most expressive features. So position wise. So here also each and every word will go independently to one one block of position wise or one one block of feed forward network. Okay. and then another layer of add on. And finally it goes to the linear layer. The linear layer maps the final output to the vocabulary space and then it goes through a soft max activation function which converts the raw scores into probability values over the entire vocabulary and then we get the output probabilities. So what exactly here what

[07:10] exactly is happening here is it is predict the decoder's work is to predict the next word correct. So as we um were telling in our last session that uh take the example of English sentence translation uh to German. Okay. So the English sentence is put as the input to the encoder. The decoder is supposed to generate one word at a time the German translation of the English words. Okay. Now when it is predicting the next word in the sequence, what happens is it will give some probability score over the entire vocabulary. So this is very important. So you have the entire vocabulary and it will give a probability score over the entire vocabulary. So and whichever word in the entire vocabulary is having the maximum

[08:06] probability. Okay. So that will come as the output probability. Okay. So that is where the soft max comes into picture. So and here we have this is basically one layer of decoder and these layers of decoder are stacked over each other. In the original paper there are six layers of decoder stacked over each other. Why this stacking is required? because it will enable the model to give much better output. Okay, because the understanding and the contextual understanding will be much better in case of stacked um encoder and decoder.

[08:46] Okay, then we will look into the attention blocks in the decoder architecture. So we have the masked multi head attention which takes the previously generated output as input okay and uses masking so that the token cannot attend to future token. So this is basically required when we are predicting the next word. So when we are predicting the next word in a sentence, what happens is you need to mask those those words which are coming and the model can have access only to those tokens which

are generated uh uh previously. Okay. So this is the reason we have the masked multi head attention because the work of the decoder is to predict the next word.

[09:38] Okay. Then we have multi head attention which is the encoder decoder based attention. And here what happens there are several heads of uh attention between the encoder and the decoder which means the queries are coming from the decoder. Keys and values are coming from the encoder. Okay. So that is where the connection between the encoder decoder is happening and the attention mechanism is playing between the encoder and decoder. And we know the attention you know score or attention weight is given by $\text{soft max of } \frac{q \cdot k^T}{d}$ where d is the dimension of the key vector into the value vector. Okay. So this is the attention uh equation for the attention with Q, K and V vectors query, key and value vectors. So this

[10:31] self attention goes inside each and every attention head and then they are concatenated to find give the uh you know final u um output. Now this allows the decoder to attend to the relevant encoder output while predicting the next token because the encoder's output we need to know or the decoder needs to know which are the relevant parts of the decoder's output which have meaningful information or important or significant information that needs to be taken into account when we are predicting the next word. Okay.

[11:11] So that is uh that is what is uh done using the multi head attention encoder decoder based attention. Okay. Then we look into the position wise feed forward network. Okay. This is ff. Okay. And uh so similar to the encoder there are two linear transformations. the ReLU activation between uh the two linear transformations same as that we have seen in the encoder. That means there are two linear transformations and the first transformation increases the dimension to a bigger dimensional space.

[11:56] Then you have the ReLU activation function. Again there is a linear transformation which reduces the dimensional space once again back and we have this f uh feed forward networks equation which is coming for the this is the for the first uh transformation. This is coming from the second transformation. This is first transformation. This is the this is these are coming from the second transformation. and max of zero and this one is coming from the relu.

[12:37] So uh relu activation function. Okay. So this we have seen in the encoder part also. So it is exactly same here also. Okay. And this uh position wise feed forward network is applied independently to each token to get a refined representation. seem similar in the encoder part as well. Now then we have the add a norm layer. So what is that one? The adder norm layer we have seen uh you know if x is the embedding vector which is the word embedding with positional encoding it goes to the sub layer whatever sub layer it is okay there are some operations sub layer of the dec of the decoder layer and then whatever operation it can be multi head mask attention it can

be uh multi head encoder decoder attention okay so there

[13:29] is a sub layer which does some operation over this um input and then whatever is coming out of that one that is your sub layer X and then the residual connection from here. So this is the residual connection which adds this one X to the output of the sub layer and then the layer normalization is done. Okay. So this is same as the encoder once again. Now the question is why does the decoder require an extra sub layer?

[14:02] So if you remember the encoder structure there were two sub layers. One was the multi head self attention another one was the position wise feed forward network. These are the two layers. Whereas in decoder part we have the masked multi head attention. Then we have multi head attention encoder decoder based. So and then we have the position wise uh feed forward network. Okay. Now why do we require this extra sub layer in decoder that is the question. So we will see for that one we have to understand what is the role that is being played by the encoder and the decoder. So the encoder it only needs to understand the input sentence. Okay. It doesn't need to know about what is getting predicted how it

[14:51] is predict predicted and so on and so forth. So its only task is to understand what is the input sequence and give the contextual understanding of that one to the decoder. Okay. Whereas the decoder it has two responsibilities. First is look at the words that are already generated. Okay. So already generated means that is the place where your masked self attention layer is coming into picture because we cannot show or show to the model the the words that are coming after that. Okay. So that that requires the masked multi head attention. Okay. Then uh there is another attention that is between the encoder's output and the between the encoder and the decoder. Okay. So the encoder has two responsibilities.

[15:42] One is look at the words which are already generated and for that one you require the masked multi head self attention and also the encoder needs to attend the encoder's output through the encoder decoder attention. So there are two places one is it has to see what are the words that are already been predicted by the decoder that responsibility is there on the decoder as well as it has to see the encoder's output because the encoder's output will give the contextual understanding where all it needs to put more focus in the encoders uh the output that is coming from the encoder side. So for that one you require another attention which is the multi head attention which is your encoder decoder based.

[16:41] Okay. And this is mast multi head self. Okay. So this is the reason why the decoder requires an extra sub layer. Now how many attention heads are there inside the multi head attention sub layer? How many attention heads? We tell that there are several heads which capture different perspective of the sentence and give a richer understanding of the sentence. Okay. Now how many attention heads are

there inside? So in the original transformer architecture eight heads are there and 16 heads are there in the transformer big model. Okay. And these heads operate in parallel within the single attention sub layer. So these all these attention heads are working in parallel and they're inside a single attention sub

[17:36] layer. So there are not you know separate sub layers for each and every attention head. So there are 16 eight or 16 attention heads that are working parallelly inside that particular um attention sub layer. Okay. Now we have the transformer architecture. Basically this u architecture in the full basically it gives how the encoder and the decoder they are connected. So we have the encoder part here with the same thing with multi head self attention add norm uh this multi-layer perceptral or feed forward network position wise add a norm and then if you see the output from here goes to the multi head attention encoder decoder based attention here. Okay. So that is where we are you know the output

[18:33] of the encoder is going into the decoder and together with that one we have the decoder's entire sequence is there embeddings plus positional encodings are going to multi head attention then add a norm multi head attention. So this in this block only this encoder's input comes into picture. Then you have add a norm here and then it goes to the um feed forward network and then add a norm once again then linear and then soft max and the probabilities. So this we have seen and this is the entire architecture of transformer with the encoder and the decoder and this is one single layer of encoder. This is one single layer of decoder. This can be multiplied n times in stacks. Okay. So in the original

[19:24] paper it is six layers of encoder and six layers of decoder. Okay. Then we come little bit we will deviate from this u you know architecture and we will see something called as the subword tokenization. Now why we are talking about subword tokenization in between the architecture encoder decoder around all the stuff because subword tokenization is used in transformers and it is a pre-processing step that splits text into smaller meaningful units and those smaller meaningful units are converted into embeddings and which are fed into the transform. transformer. So it is a pre-processing step that is required before you put the embedding to the transformer. Okay. So in this category we have two types of subword

[20:21] tokenization. One is the bite pair encoding. Another one is the word ppiece tokenization. So we will look into these two in my subsequent slides. So bite pair encoding what does it uh do? It breaks the words into smaller frequently occurring pieces subwords. Okay. So we will see in an example it sees how what with what frequency those type of subwords or you know characters are coming together and based on that one the encoding is done.

[20:55] Okay. So we will look into that one. Whereas in wordpiece tokenization it breaks the words into smaller meaningful pieces. Okay. So depending on the likeli-

hood of two words coming together, the word piece tokenization breaks the words into smaller meaningful pieces. Okay. Now why do we use subword tokenization? Tokenization is a part of pre-processing a very standard part of NLP pre-processing technique. We know because tokenization is required because by that each and every word meaning is embedded into the embedding vector and then it goes to the model. Now why is this subword tokenization important here? Subword tokenization have the advantage of with this one we can handle rare or unseen words. Okay.

[21:52] Suppose you have the word you know cyber security then that can be broken into cyber plus security. Okay. So if the model knows the meaning of cyber and the model knows the meaning of security then it can frame the meaning of cyber security. So that is the beauty of subword tokenization. So we will look into what is this bite pair encoding and what is this wordpiece tokenization with some example. So in bite pair encoding suppose you have a training corpus with the words low lower and newer. So this is your corpus training corpus. Okay. And then you split the words into characters. So what are those characters? L O W L O W E R L O W E R S T N E W N E W E R New Okay. So these are split into the characters.

[22:54] Okay. Now we will see what is the most frequent pair and then we will merge it. So if you see the most frequent pair is L O L O L O okay see here there are three places L is coming we is coming in two places okay fine but from here you can see L is the most frequently coming you know uh pair now we merge them together so L + O we get L O. Okay. Now we join those two everywhere here. Wherever we got it L then W. L then W E R L O then W E S T.

[23:48] Okay. So wherever we are getting L we have merged them and remaining part we have kept it as it is. Okay. Then it can be written as L + W. It is gives you low. Okay. Can I write it like this? So then L + W E R gives lower. Okay. So like that. So here we have L low L E R L O W E S. What we have done is after we have found Lo then the next most frequently paired uh uh character is W. That is the reason I have added W here and made it low because everywhere W is coming after L.

[24:37] Okay. So now I have L W L W E R L O W E S T. Okay. So this is my merged part. So what we have done we had the training comp corpus low lower newer then I have split the words into characters all the characters are separated out and then I have found the most frequent pair. So the most frequent pair is L and O together. So here here you are getting that I have joined together merged together and I have written the other part like this. Okay, whatever is wer e s and then I have tried to find out what is the next character which is coming more frequently which is your w. So l plus w gives low and then you have written all these words in terms of low.

[25:33] Low lowest. Okay. So the vocabulary now becomes low. Low is common. E r e s t and new. Why this new is there? Why this new ER is not there? Because you have already you have this ER. So if you add new plus ER, it becomes newer. You

don't need to keep the newer another separate word here. So the vocabulary size can be reduced by reusing these smaller pieces subwords. Okay. So this is the bite pair encoding. So the learn and vocabulary now stores subwords instead of complete words. Okay.

[26:22] Then we will look into word piece tokenization. What exactly it is doing? So suppose we have a vocabulary play in or ed. So this hash terms basically means there is something before this is not a new word. So playing or player or played or singing singer okay like that. So this is this is the vocabulary. Now this playing term can be written as play plus in player can be play can be written as play plus e ed e d. Player can be written as play plus e r. Okay.

[27:03] So this is the way you can write the big words with the subwords. Okay, these are called the word piece tokens. And with this one we can as well we have told it before also that we can handle rare or unseen words like microscope. So micro plus scope. If the model already knows the meaning of the word micro and scope, it combines them together to you know understand the complete uh sentence together with that means like that another example can be unhappiness. So un plus happiness. Okay. So that those two subwords can be joined and this un can be you know used with other words as well to give you know meaningful um words. Okay.

[27:57] Now, so what is the advantage of this uh subword tokenization? It is basically instead of memorizing every complete word, the model learns a smaller set of reusable words. Okay. So this helps in reducing the memory size and thus the subword tokenization helps in you know computational efficiency. Okay. And what piece chooses margins based on likelihood unlike bite pair encoding which uses which uses frequency which uses frequency okay so [snorts] that's about bite pair encoding and wordpiece tokenization okay then we will look into the training of the transformer so we have shifted a little bit from the architecture of the transformer because we wanted to know what kind of pre-processing is done before the

[28:58] embeddings are put into the transformer architecture. That is where our bite pair encoding and WP tokenization came into picture. Then we are back into the training of the transformer. So in the training of the transformer the data set used was the original work it is WMT 2014 English to German translation data set. that they have used as the data set and it contained approximately 4.1 sentence pairs. What do you mean by sentence pair? That means 4.5 English sentences and their German translation. So that was the data set that is used. Okay. It also was trained with English to French data set WMT 2014 English to French data set and this data set is much larger than the English to German translation

[29:47] data set and it consisted approximately 36 million sentence pair that means English and French. Sentence number one English, sentence number one French, sentence number two English, sentence number two French. So that entire pair 36

million sentence pairs are there in that data set. So that is used for training the transformer. The English to German model use the bite pair encoding and English to French model use the wordpiece tokenization.

[30:18] Okay. Now then we will before concluding we will have an look into the advantage and limitations. So the advantage of transformer architecture is very you know obvious. It is uh you know parallel computation is possible. Better long range dependency modeling is possible. It is highly scalable state-of-the-art performance and it is the foundation of for modern large language models like bird GPT which we will see in our subsequent lectures. Okay. And what are the limitations of this one? Okay. Of course, parallel computation is possible, highly scalable, but the memory usage is high. Quadratic attention complexity means the if you have n number of tokens, the number of operations will go as n square. Okay,

[31:06] requires larger data sets to be trained and it is computationally expensive. So these are the limitations of the transformer architecture. Okay, but they are the state-of-the-art and with this one I come to the end of today's session and um in my next session we will be talking about the birth model. Okay. And uh before I conclude today's session we will quickly have a recap of what all we have learned in this session. So we have learned about the decoder architecture in the transformer and we have seen that decoder architecture is having you know six layers of stacked decoder. Each of these layers have three sub layers. One is the masked multi head attention which is required because the decoder's work

[31:56] is to predict the next word in a sentence. So the masking masked multi head attention is very much required which enables the model only to look into the tokens that are generated and masks the uh future tokens that are coming into picture uh you know uh later. Now there is another attention block which is called as the multi head attention between the encoder and the decoder. Okay. So that is required because you need to understand what is the encoder's output to predict the next word which is the main function of the decoder. Okay. So that is the encoder decoder's multi head attention. Then we have the position wise feed forward network. Okay. And in between we have this add and nom layers and you know it

[32:46] is the other things are similar to the encoder and we have the residual connections and then we have the normalization layers which helps in getting training stability and uh the residual connections helps to keep the uh original information intact. Okay. So with this the entire architecture of decoder we have seen and then we have shifted a little we have seen how the encoder and decoder are connected together in the in the complete um transformer architecture. The output of the encoder goes in the multi head encoder decoder attention block. That is where the encoder's input comes into picture. Okay. And then we have seen what is the subword tokenization, how bite pair encoding uh works and how wordpiece tokenization

[33:39] works. How why subword tokenization is required. It is one of the pre-processing steps before you can feed your uh embeddings into the transformer architecture. And we have seen with these subword tokenization techniques like bite pair and word piece uh the vocabulary can have a lot of reusable words instead of keeping separate separate completed words that helps in reducing the vocabulary size which helps in the memory requirement of the um model. Okay. So and then we have looked into the training what are the data sets that have been used what are the type of encodings uh uh subword uh tokenizations that has been used for uh pre-processing uh in those two data sets and then we have looked into the advantages and

[34:29] limitations of the transformer architecture as a whole. Okay. So there are a lot of advantages scalability, parallelization, long-term dependency capture effectively. But there are limitations like it is computationally heavy and then um you require larger data sets and you know your um computation goes in quadratic way the computation increases number of operations increases and uh so these are the main limitations of uh transformer architecture. So with this one I come to the end of today's session. So in the next session we will be talking about the language model. The bird language model we will be discussing about. Till that time happy learning and thank you all.